AccessGuard-AI

Automated Barrier Scanner for Public Digital Services

Problem and scope

Digital services can exclude people with disabilities when pages contain low-contrast text, missing image alternatives, unlabeled form controls, or other accessibility barriers. Teams also need findings explained in language that helps them prioritize practical fixes. AccessGuard-AI scans a user-supplied public website with a real browser-based accessibility engine and presents evidence-backed findings rather than inventing results or claiming full WCAG compliance from an automated score.

What it does

• Authenticated users can submit a website URL for a browser-based audit using Playwright and axe-core.
• Findings include rule identifiers, impact/severity, affected element evidence and remediation guidance.
• A React dashboard supports scan history and detailed inspection; the interface includes Citizen Mode (English/Kannada voice support), AccessVoice, and Senior Safety Shield features.
• Scan results support JSON/CSV and SARIF export; WCAG-oriented scoring/principle summaries are intended to organize findings, not certify conformance.
• Backend protections validate target URLs and block private/loopback/link-local and cloud-metadata destinations to reduce SSRF risk.

Architecture

React + Vite client Login • Dashboard • Scan details • Citizen/voice UI
HTTPS / JSON API • authenticated requests
Node.js + Express API Auth • scan routes • URL/SSRF validation • exports
Playwright browser + axe-core audit engine
SQLite Users • scans • violations

Implementation and run

Stack: Node.js/Express, React 18, Vite, SQLite, Playwright, axe-core, JWT/cookie-based authentication, and automated tests. Requirements: Node.js 20+ and npm 10+. From a clean checkout, install dependencies in the root/client/server as specified by the repository package files, install the Playwright Chromium browser (`npx playwright install chromium` from the server directory), configure local environment variables using the supplied `.env.example`, then start the backend and frontend using the README scripts. Confirm exact scripts and environment requirements against the committed README before judging or demoing.

Evidence, status & limitations

Hackurity 2026 • Repository: github.com/gowtham-cr/AccessGuard-AI

Core deliverables status

Deliverable	Status / where to verify
Real accessibility scan	Implemented; verify by running a scan against a permitted public test target and inspecting the saved result.
Authentication and scan persistence	Implemented in backend; test registration/login, protected scan APIs, and scan history locally.
WCAG guidance and severity/detail view	Implemented; inspect a scan detail page and compare displayed rule IDs/evidence to the axe-core result.
CSV/JSON and SARIF exports	Export code and tests are present; verify downloaded files and SARIF structure in the live demo.
Citizen Mode, AccessVoice, Senior Safety Shield	UI/features are present; demonstrate only flows verified on the submission machine.
AI-powered remediation	Optional and credential-dependent as described in README; do not claim it ran unless tested with a valid key. Rule-based guidance is not represented as AI.
Human accessibility evaluation	Not replaced by automation; keyboard, screen-reader and manual WCAG checks remain necessary.

Threat and user model

Users are developers and non-specialist teams reviewing accessibility of public digital services. The scan URL is untrusted input. Risks include SSRF into local/private services, credential exposure, cross-user scan access, misleading compliance claims, and unsafe reliance on generated advice. Defenses include URL/protocol validation, DNS/IP-range checks, authentication, per-user scan ownership, server-side secret handling, and evidence-based rule guidance. Only scan sites for which the team has permission; use no real credentials or personal data.

Assumptions, limitations and next steps

Automated axe-core checks detect only a subset of accessibility issues and cannot establish complete WCAG conformance. Results depend on page state, reachable content, authentication requirements, and browser behavior. Recheck SSRF defenses against redirects and dynamically loaded resources; verify every feature on a clean machine; manually test keyboard and screen-reader use; document any partial or unverified features honestly.

Evidence and measurement

The project team reported a frontend test run of 45 passing tests across 5 files and a production build success. The backend suite was reported as 121 passing tests across 26 suites by the coding assistant; this figure should be included as a reported result only after rerunning and confirming it locally. Do not claim a measured accessibility pass rate or WCAG compliance percentage. During the demo, show the target URL, real scan output, at least one finding with element evidence, and one exported report.

Credits and AI-tool disclosure

Core technologies include React, Vite, Express, SQLite, Playwright, axe-core, and associated project dependencies; see package manifests and their licenses. The project README/write-up should explicitly name the AI coding assistant used (Antigravity/AI-assisted coding, if accurate), describe its role in code generation/debugging/UI iteration, and identify any reused code, assets, or datasets. The glass background asset should be credited if sourced externally; if generated by an AI tool, disclose that. Add a LICENSE file before final submission if not already present.

Repository and submission

Source repository: https://github.com/gowtham-cr/AccessGuard-AI
Submission form: https://forms.gle/wsCxj9oUmJuknaCEA
Before submitting, confirm the repository is public, the `submission` tag exists remotely, the README is corrected to match the actual project, the PDF is uploaded to a shareable URL, and the demo video is no longer than 5 minutes and accessible without login.

Prepared from the project information available at submission time. Confirm all statuses, test counts, credits and run instructions against the committed code before submission.